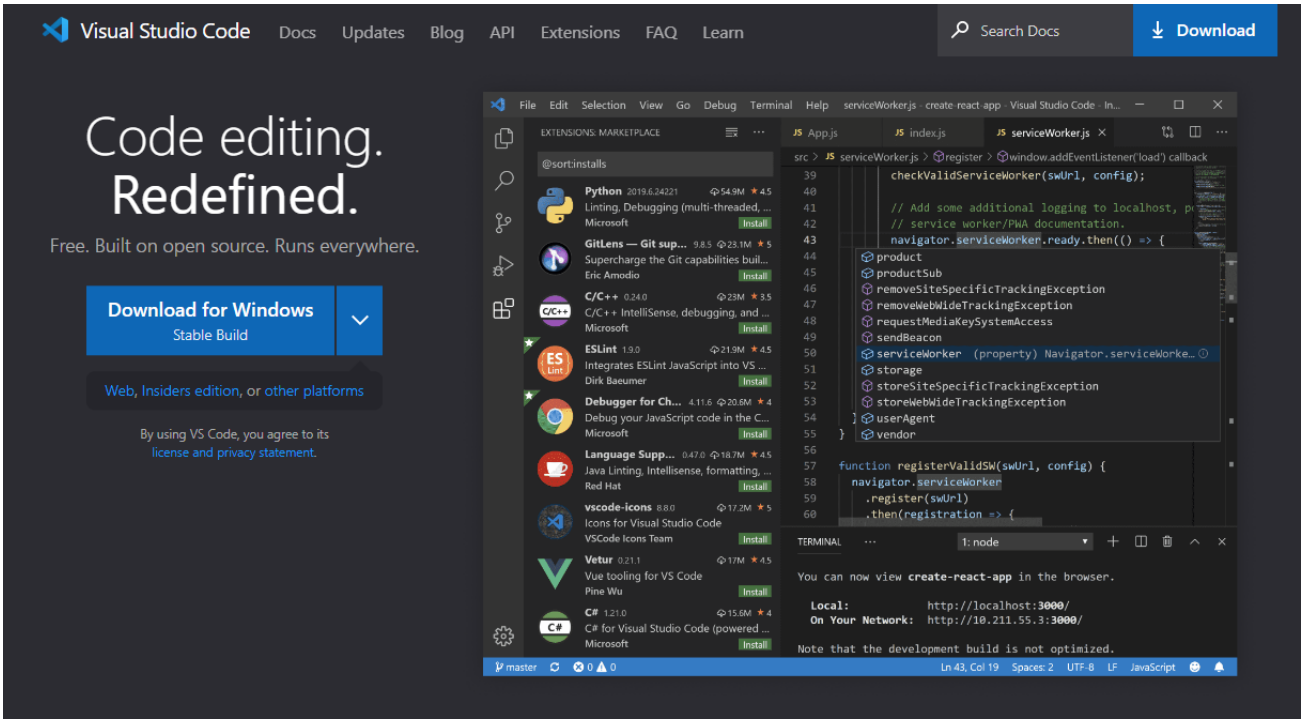
vscode配置C++以及boost库开发环境

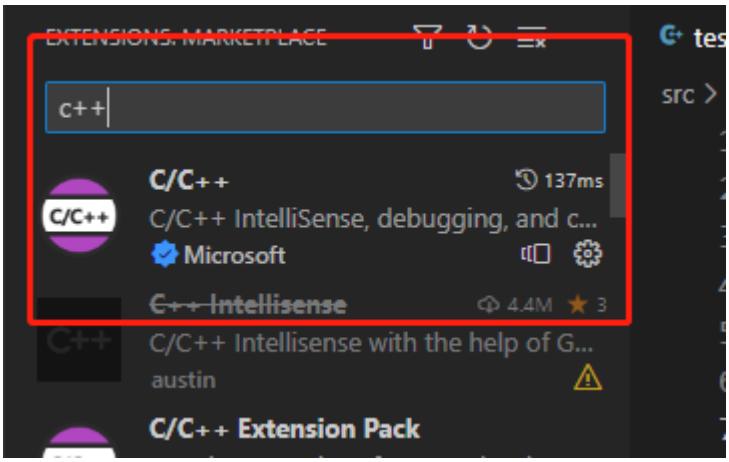
Posted on 2023-02-01 15:07:23 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2L7w5DaotbULpN3UsUN9PlGyCvP)

下载vscode

去官网https://code.visualstudio.com/ (https://code.visualstudio.com/)下载最新版本就可以了



下载后打开vscode，安装C++扩展



目前就先配置到这里，以后的配置留给下面介绍。

下载mingw

去mingw官网下载压缩包
https://sourceforge.net/projects/mingw-w64/files/ (https://sourceforge.net/projects/mingw-w64/files/)
下载速度慢的话可以去我的网盘下载，整理了mingw和boost的安装包

链接: https://pan.baidu.com/s/1IJSyQqdf-IbuFG12FhDt2A?pwd=2dhi (https://pan.baidu.com/s/1IJSyQqdf-IbuFG12FhDt2A?pwd=2dhi)
提取码: 2dhi

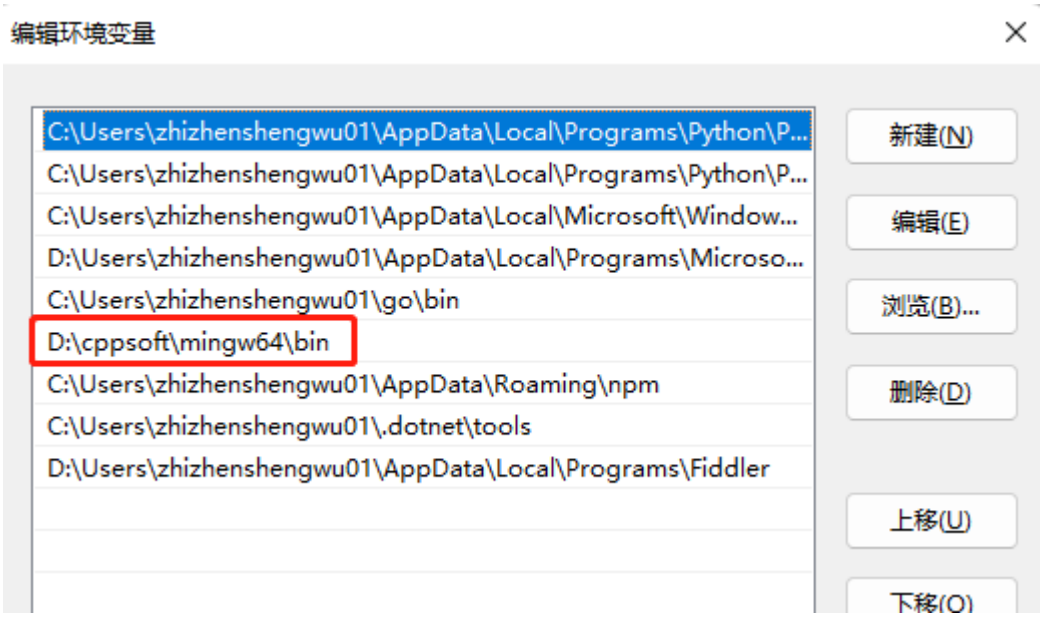
选择x86_64-win32-seh,当然选择x86_64-posix-seh也可以。一个是windows线程，一个是posix线程。



下载后解压缩到指定文件夹，我的目录为 D:\cppsoft\mingw64



配置环境变量，选择用户的环境变量path, 添加D:\cppsoft\mingw64\bin，具体路径看你自己的配置



测试是否配置成功，打开cmd，在里面输入g++或者gcc，如果输出no input files 则说明配置成功

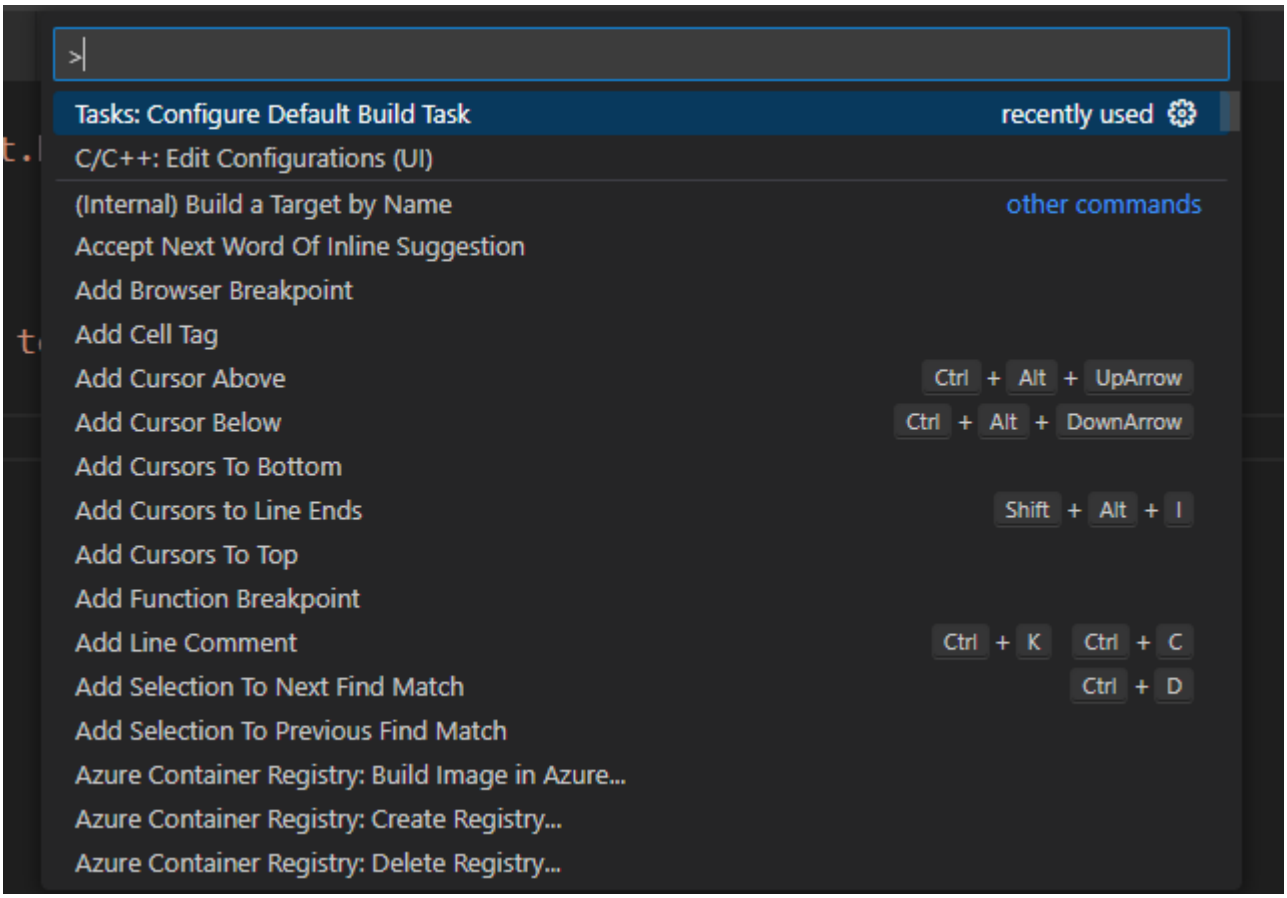


配置vscode C++开发环境

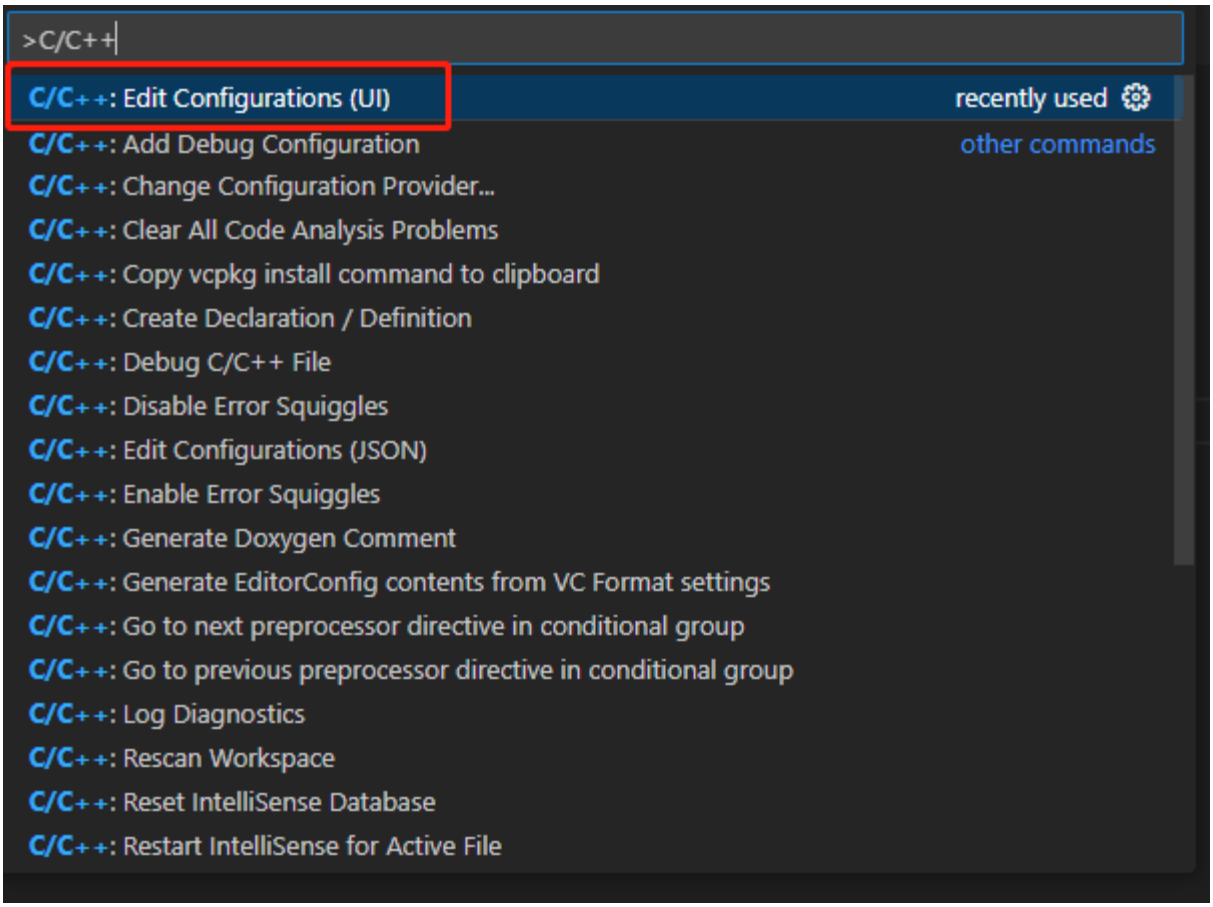
主要分为配置编译器信息，配置构建任务task.json以及配置调试设置lauch.json文件三个步骤。

配置编译器信息

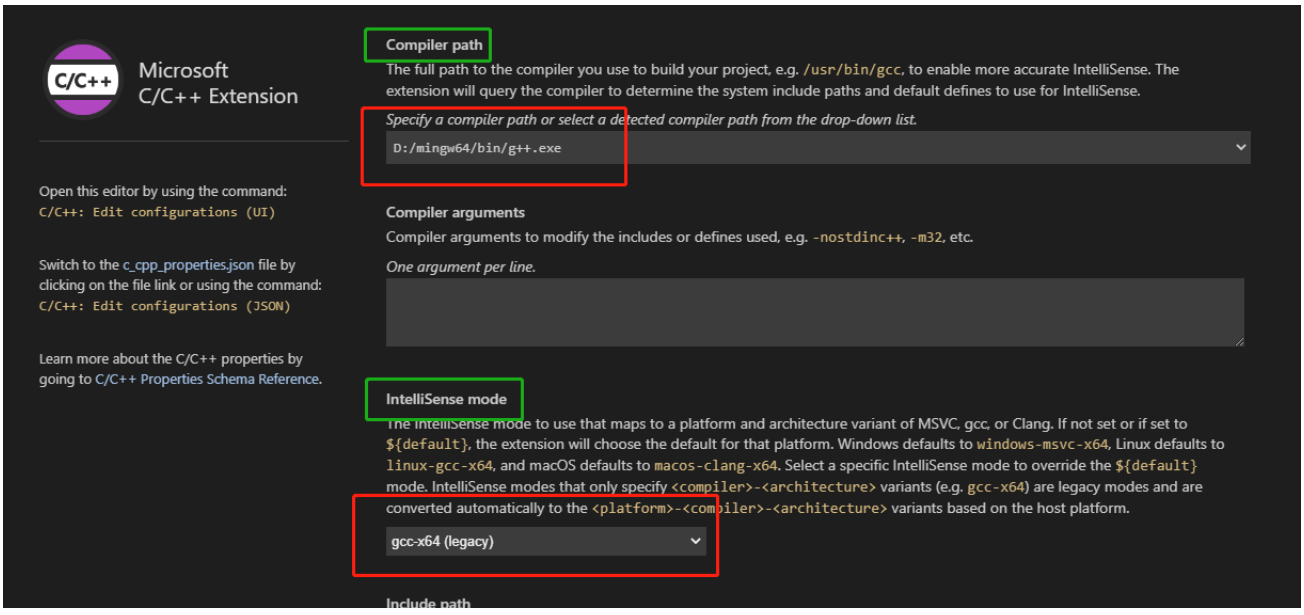
在vscode界面按住Ctrl + Shift + P 唤出配置界面



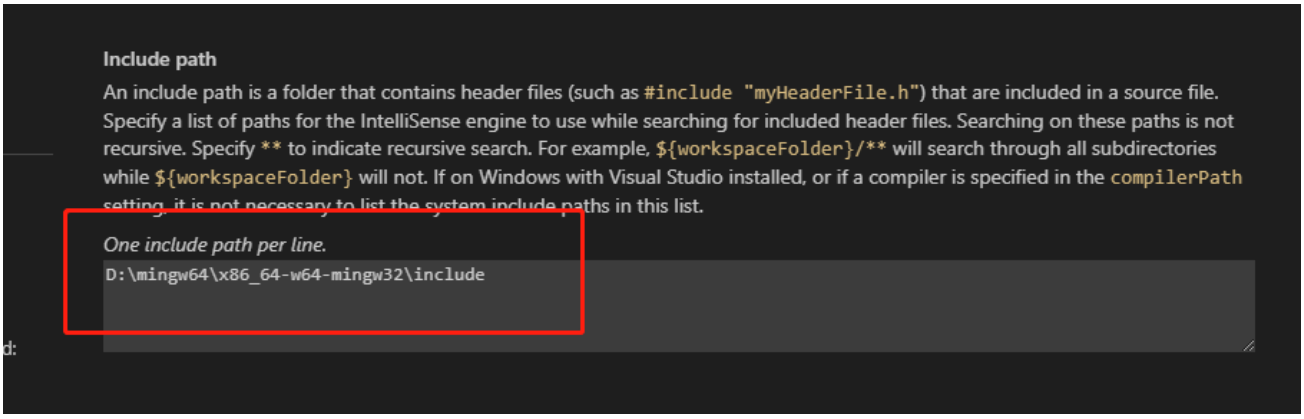
在输入栏输入C/C+， 选择“Edit Configurations(UI)”进入配置



配置编译器路径和IntelliSense模式,我的路径是D:/mingw64/bin/g++.exe

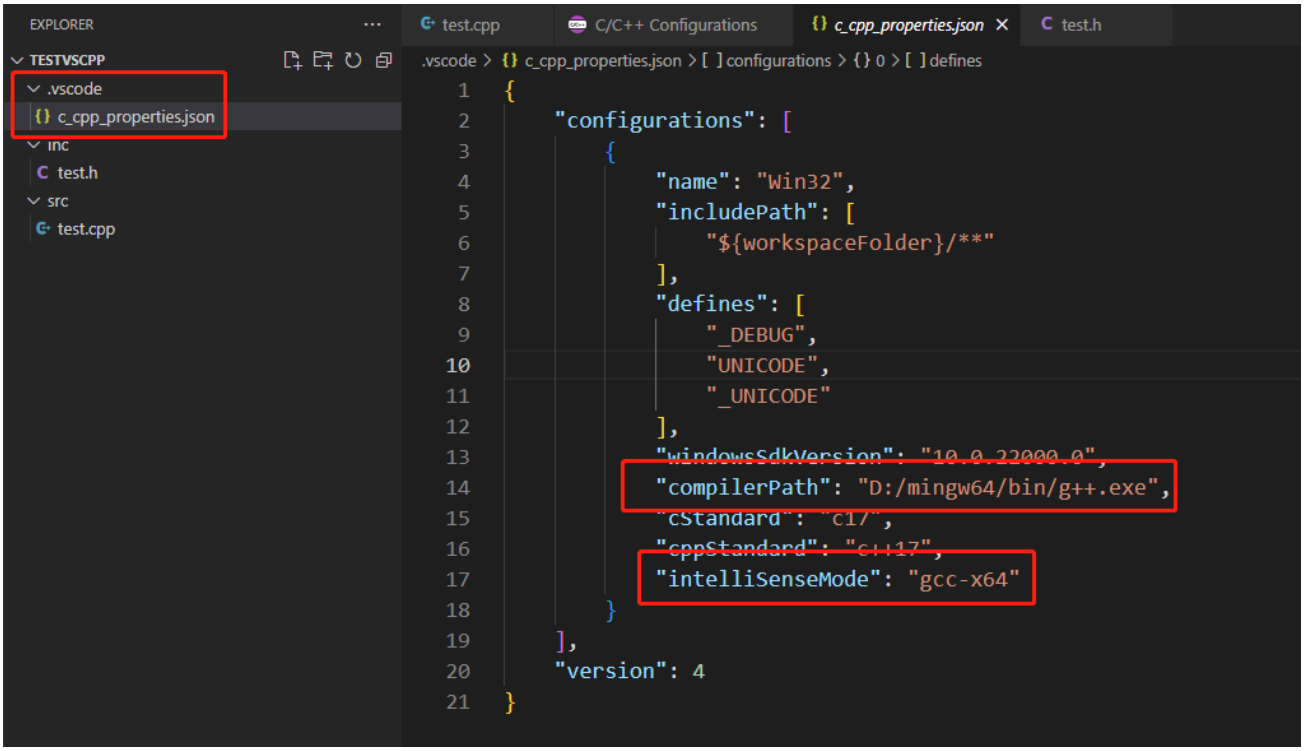


另外要注意配置Include path,其他人的博客很少提及这点，这里着重说一下



上述情况配置好include路径后，包含第三方库才不会报错，否则会有错误提示。

ctrl+s保存后会生成一个.vscode文件夹，.vscode文件夹里有一个c_cpp_properties.json文件，记录了我们配置的编译器信息。

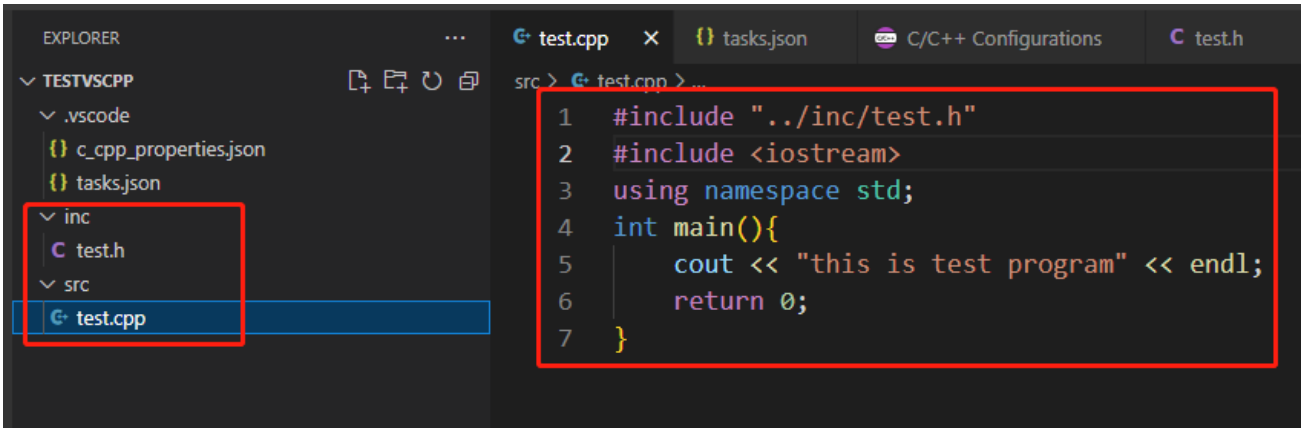


json文件信息如下

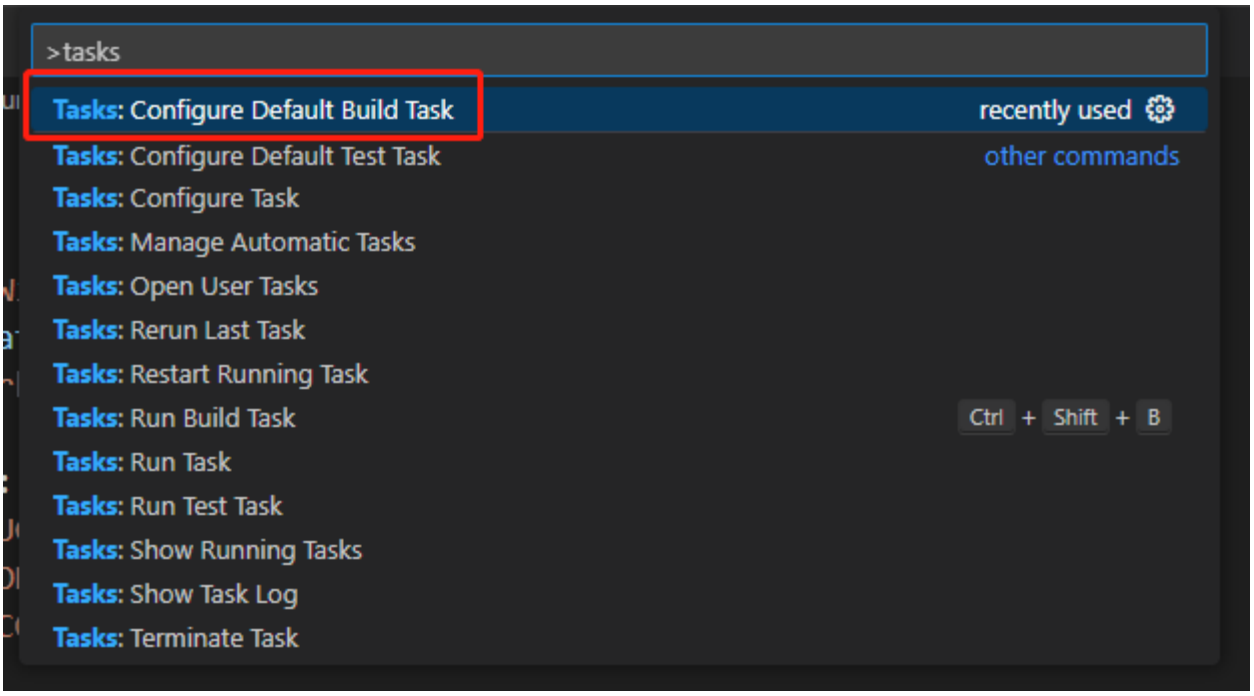
```
{
    "configurations": [
        {
            "name": "Win32",
            "includePath": [
                "${workspaceFolder}/**"
            ],
            "defines": [
                "_DEBUG",
                "UNICODE",
                "_UNICODE"
            ],
            "windowsSdkVersion": "10.0.22000.0",
            "compilerPath": "D:/mingw64/bin/g++.exe",
            "cStandard": "c17",
            "cppStandard": "c++17",
            "intelliSenseMode": "gcc-x64"
        }
    ],
    "version": 4
}
```

配置构建任务

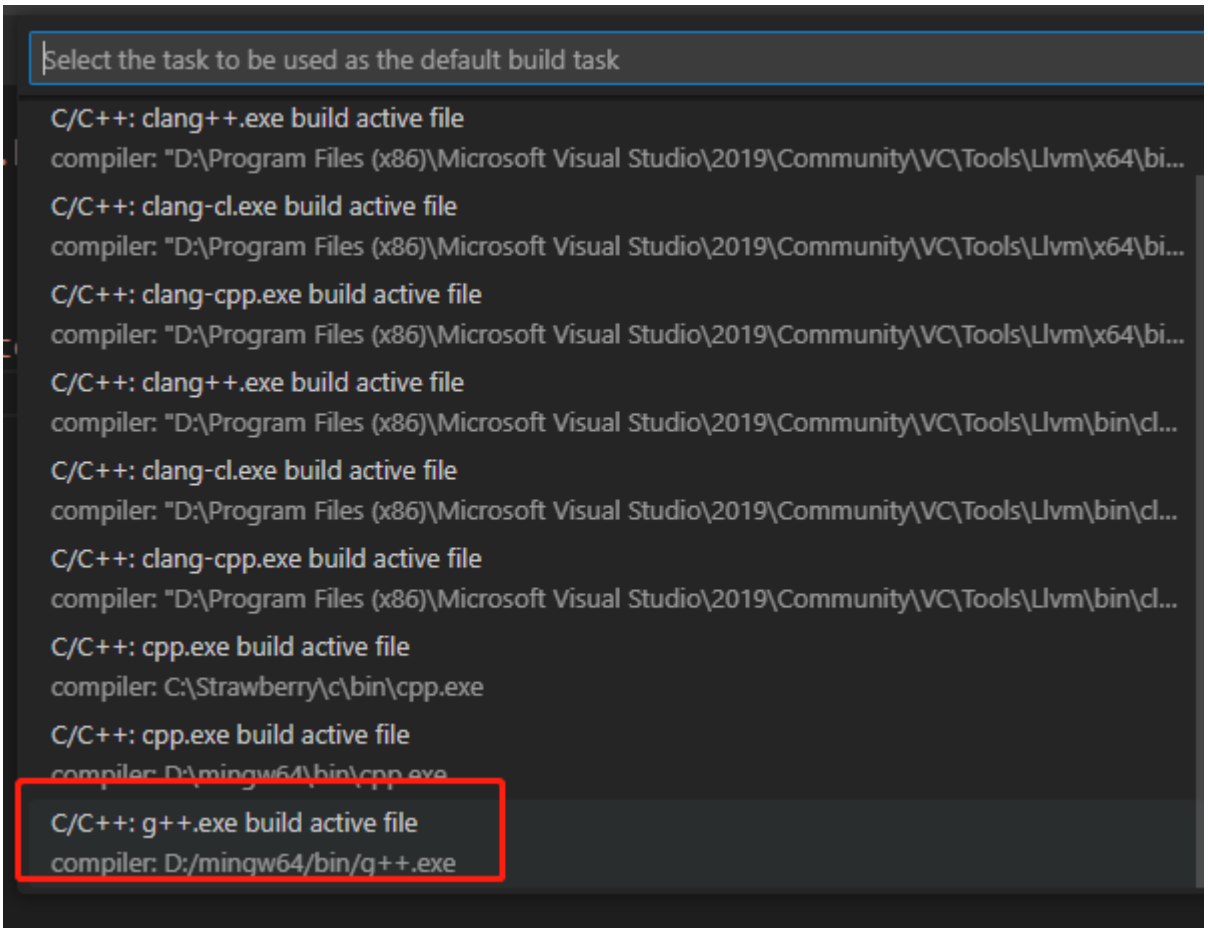
配置任务之前我们先写一个.cpp和.h文件用来测试，组织形式如下



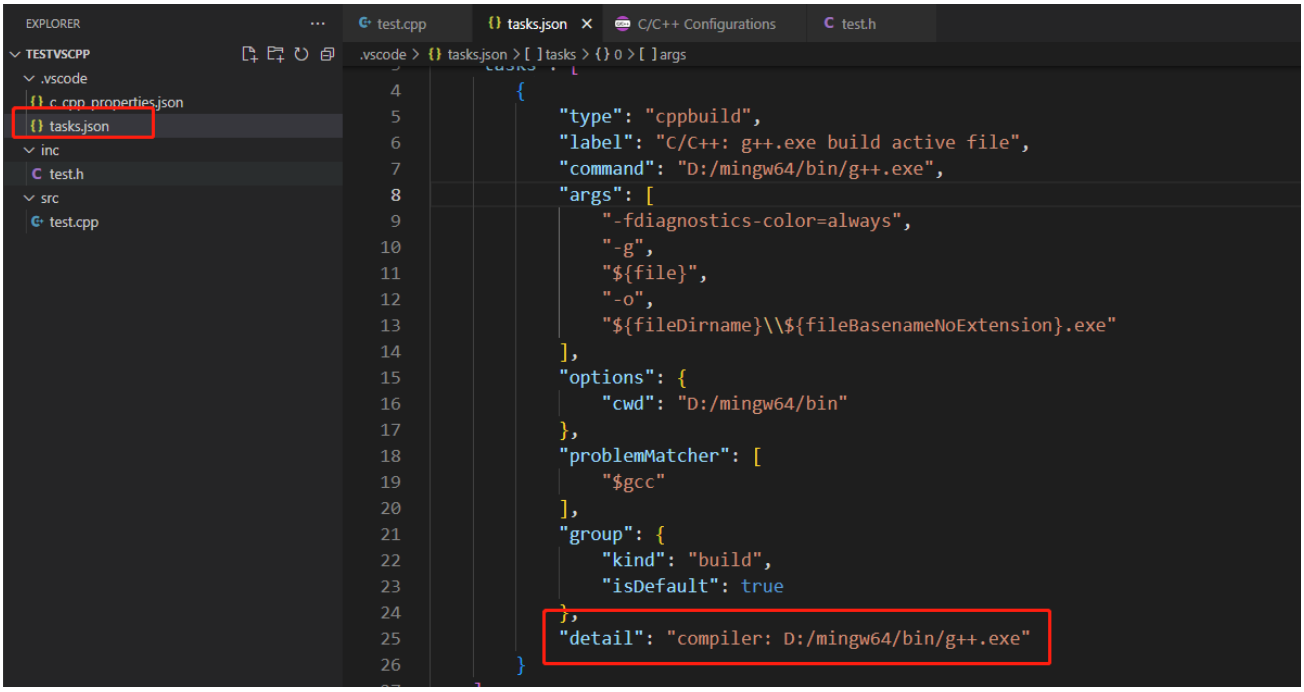
然后选中左侧目录树的cpp文件，保证我们所有的配置都是基于cpp构建的。
接下来，创建一个tasks.json文件来告诉VS Code如何构建（编译）程序。该任务将调用g++编译器基于源代码创建可执行文件。 按快捷键Ctrl+Shift+P调出命令面板，输入tasks，选择“Tasks:Configure Default Build Task”：



可以看到vscode列举出我的电脑已经安装的所有C++编译器，这里选择
再选择“C/C++: g++.exe build active file”,这是我们刚刚安装的mingw编译器



点击后vscode为我们自动生成了task.json文件

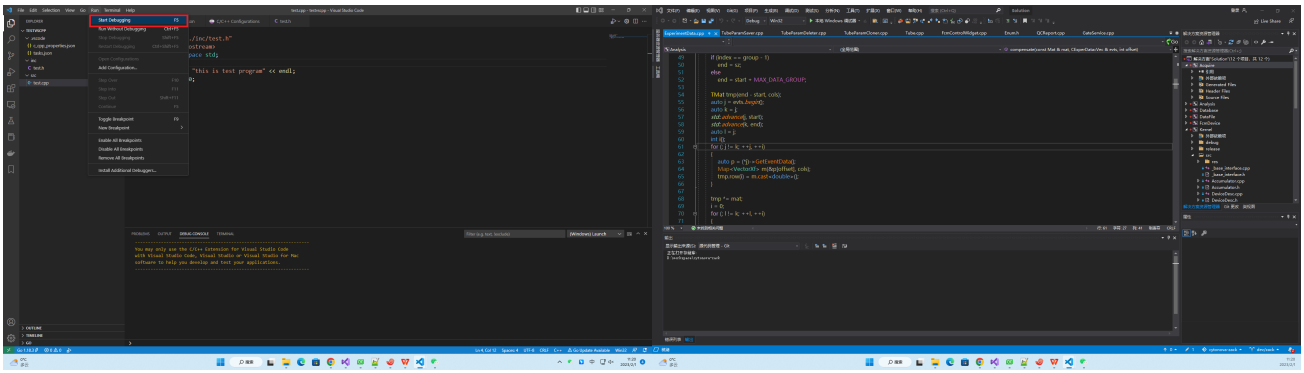


详细的task.json配置信息如下

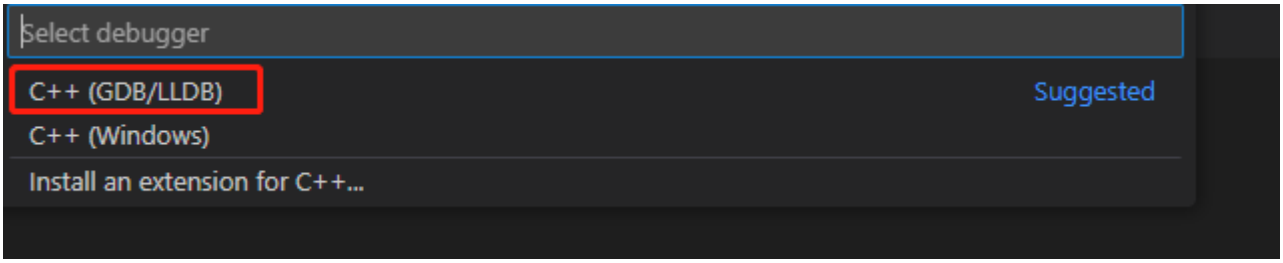
```
{
    "version": "2.0.0",
    "tasks": [
        {
            "type": "cppbuild",
            "label": "C/C++: g++.exe build active file",
            "command": "D:/mingw64/bin/g++.exe",
            "args": [
                "-fdiagnostics-color=always",
                "-g",
                "${file}",
                "-o",
                "${fileDirname}\\${fileBasenameNoExtension}.exe"
            ],
            "options": {
                "cwd": "D:/mingw64/bin"
            },
            "problemMatcher": [
                "$gcc"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "detail": "compiler: D:/mingw64/bin/g++.exe"
        }
    ]
}
```

配置调试设置

同样选中左侧目录树的cpp文件，保证我们所有的配置都是基于cpp构建的。
这里主要是为了在.vscode文件夹中产生一个launch.json文件，用来配置调试的相关信息。点击菜单栏的Debug—>Start Debugging：



接下来选择C++(GDB/LLDB), 当然你选择windows的也可以。



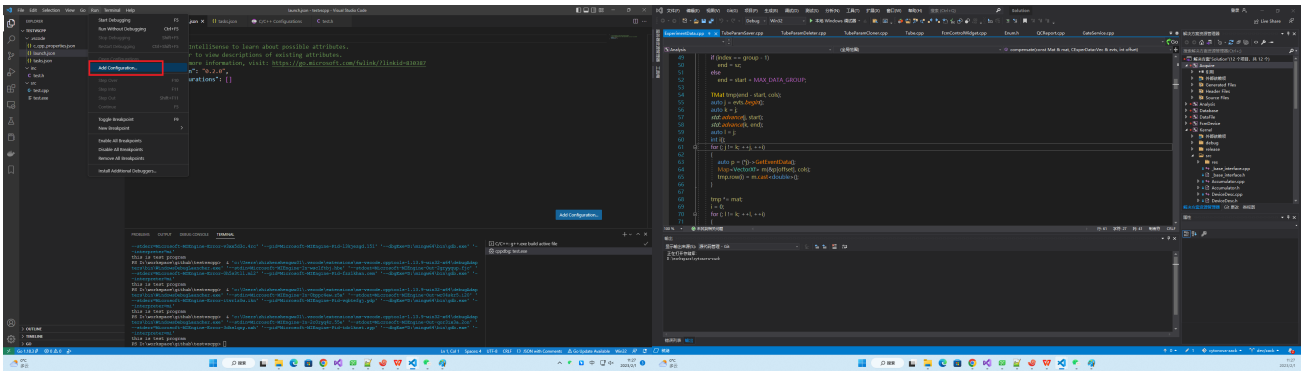
紧接着会产生一个launch.json的文件。

这里有的人没有生成launch.json文件，而是直接运行成功了，可以不配置launch.json文件了。

如果生成launch.json文件，则如下

```
{
    // Use IntelliSense to learn about possible attributes.
    // Hover to view descriptions of existing attributes.
    // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
    "version": "0.2.0",
    "configurations": []
}
```

如果想生成launch.json还有一个办法，就是点击Run选择Add Configuration, 就可以生成launch.json文件了。



关于launch.json文件的配置可以参考官方文档，大家可以直接用我的配置如下

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "g++.exe build and debug active file",
            "type": "cppdbg",
            "request": "launch",
            "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": true, //修改此项，让其弹出终端
            "MIMode": "gdb",
            "miDebuggerPath": "D:\\cppsoft\\mingw64\\bin\\gdb.exe",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ],
            "preLaunchTask": "task g++" //修改此项
        }
    ]
}
```

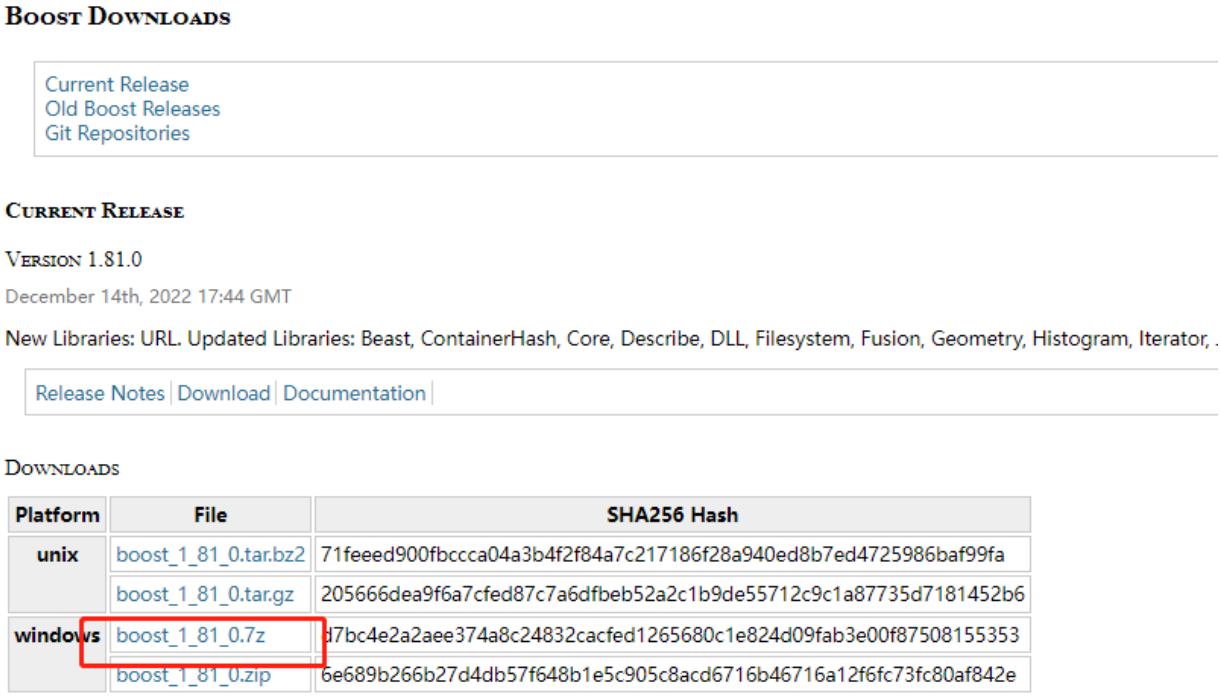
miDebuggerPath 改为你自己的mingw路径
preLaunchTask 为任务名字，你可以自己定义一个任务名字，任务名字要和task.json中的label相匹配。我的名字为”task g++”，那么我会将task.json中的label也改为”task g++”。我的task.json文件如下

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "type": "cppbuild",
            "label": "task g++",
            "command": "D:/mingw64/bin/g++.exe",
            "args": [
                "-fdiagnostics-color=always",
                "-g",
                "${cwd}//src//*.cpp",
                "-o",
                "${fileDirname}\\${fileBasenameNoExtension}.exe"
            ],
            "options": {
                "cwd": "D:/mingw64/bin"
            },
            "problemMatcher": [
                "$gcc"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "detail": "compiler: D:/mingw64/bin/g++.exe"
        }
    ]
}
```

- 我们修改了label为”task g++”
- 我们修改了args中的-g选项为”${cwd}//src//*.cpp”
  选择test.cpp再次运行，程序通过。

安装boost库

先去官网下载boost库最新版本
https://www.boost.org/users/download/ (https://www.boost.org/users/download/)
选择windows版本下载，zip和7z格式的都可以



Boost Downloads

Current Release
Old Boost Releases
Git Repositories

Current Release

Version 1.81.0

December 14th, 2022 17:44 GMT

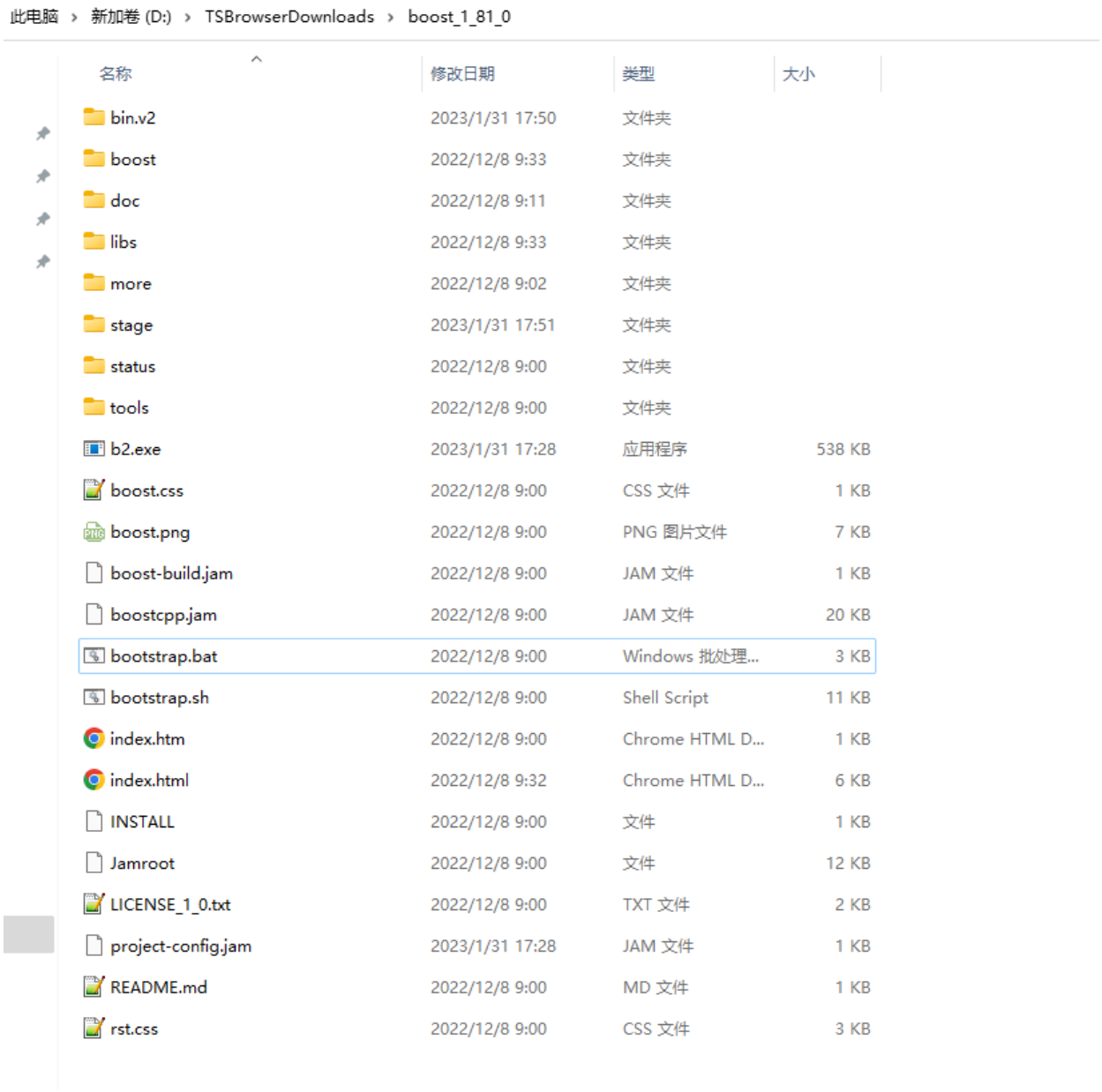
New Libraries: URL. Updated Libraries: Beast, ContainerHash, Core, Describe, DLL, Filesystem, Fusion, Geometry, Histogram, Iterator, .

Release Notes | Download | Documentation

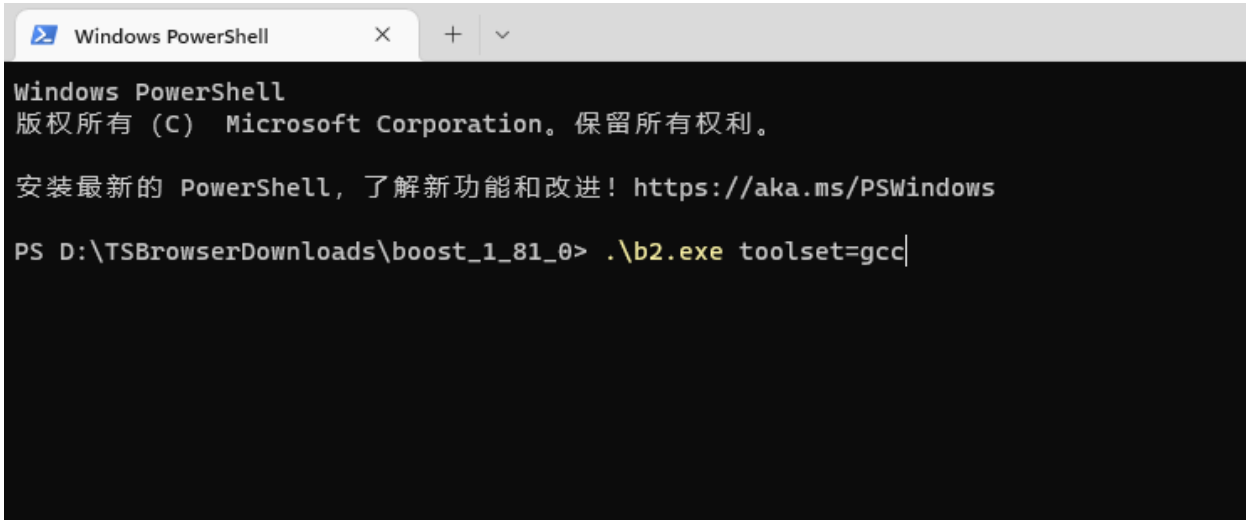
Downloads

Platform	File	SHA256 Hash
unix	boost_1_81_0.tar.bz2	71feeed900fbccca04a3b4f2f84a7c217186f28a940ed8b7ed4725986baf99fa
	boost_1_81_0.tar.gz	205666dea9f6a7cfed87c7a6dfbeb52a2c1b9de55712c9c1a87735d7181452b6
windows	boost_1_81_0.7z	d7bc4e2a2aee374a8c24832cacfed1265680c1e824d09fab3e00f87508155353
	boost_1_81_0.zip	6e689b266b27d4db57f648b1e5c905c8acd6716b46716a12f6fc73fc80af842e

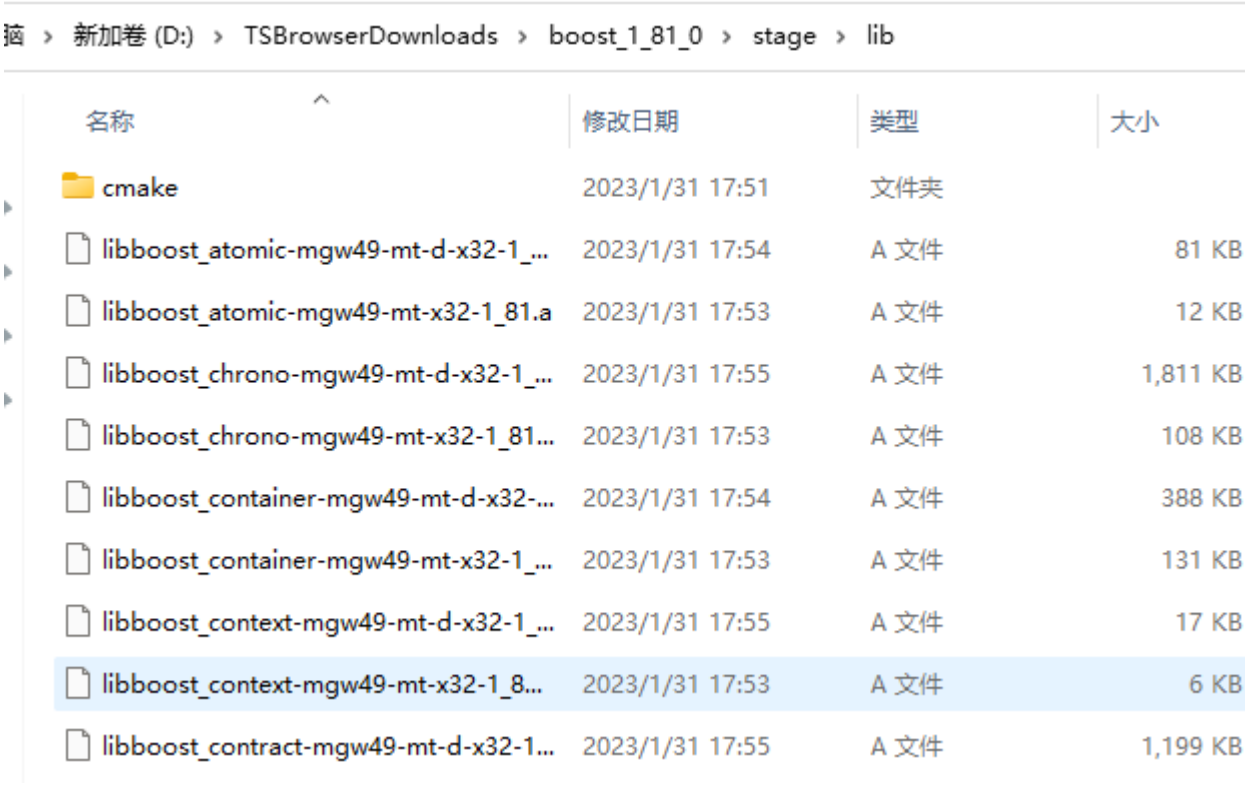
解压后文件夹下有个一个bootstrap.bat文件，双击运行会生成b2.exe



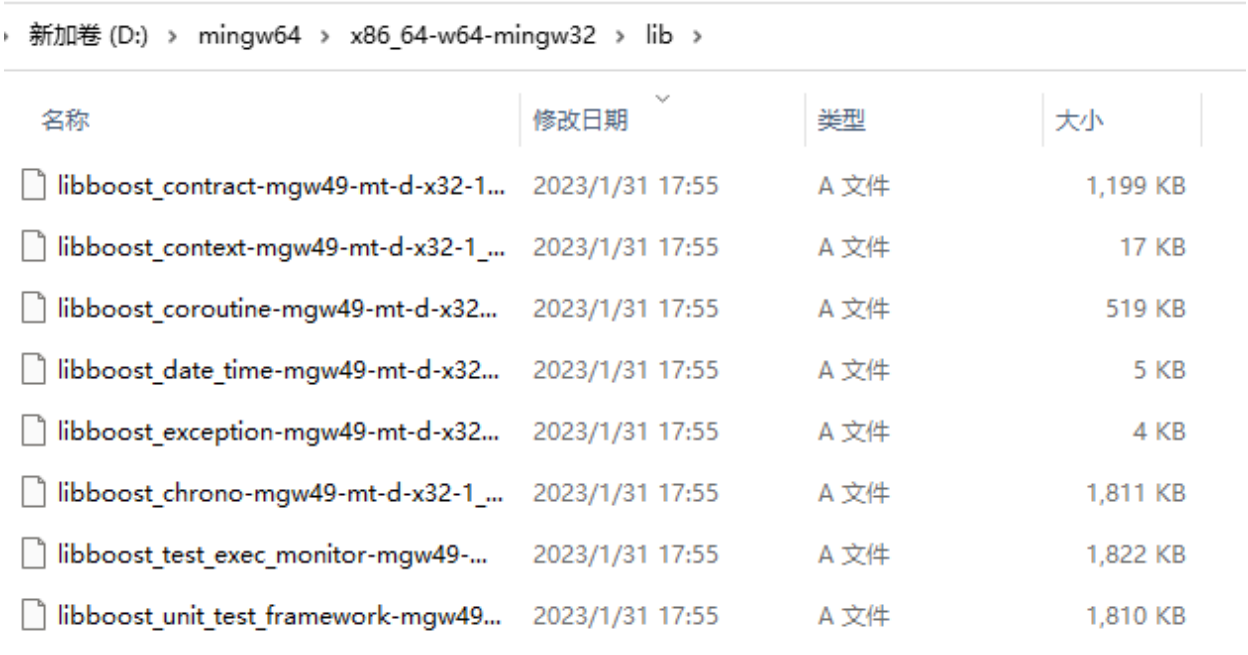
然后在boost文件夹下启动cmd，执行 “.\b2.exe toolset=gcc”



编译时间和机器性能有关，执行编译过后，会在stage文件夹下生成lib文件夹，里面就是我们要用到的lib库。

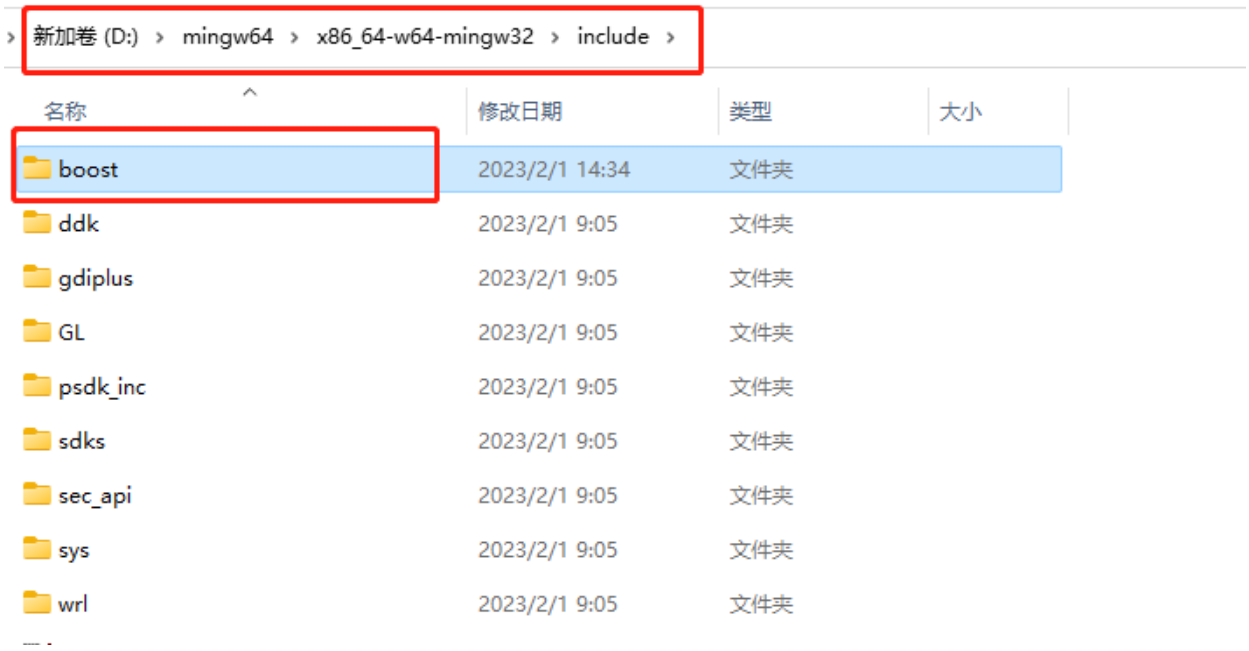


接下来需要将boost文件夹下的staget文件夹下的lib文件夹下的库放到mingw的文件夹下x86_64-w64-mingw32的文件夹下的lib文件夹内。
我的目录就是从”D:\TSBrowserDownloads\boost_1_81_0\stage\lib”复制到”D:\mingw64\x86_64-w64-mingw32\lib”。



同时也要把boost的头文件从boost文件夹下拷贝到x86_64-w64-mingw32的include文件夹下
也就是从”D:\TSBrowserDownloads\boost_1_81_0\boost”拷贝到D:\mingw64\x86_64-w64-

mingw32\include



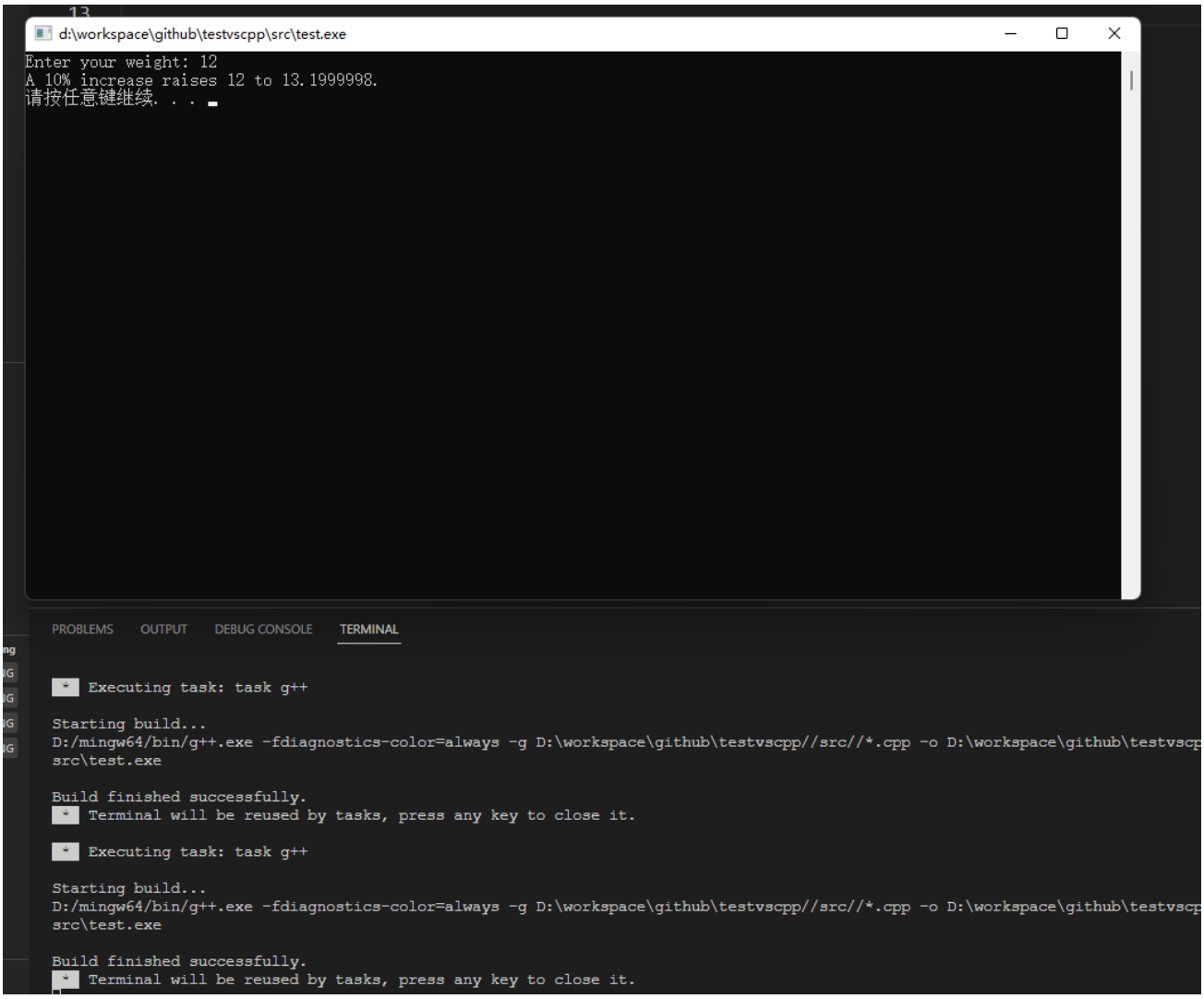
测试boost库

需要对已经配置完成的boost库进行测试，我们写一个程序, 替换原来的test.cpp的main函数

```
#include <iostream>
#include <string>
#include "boost/lexical_cast.hpp"

int main()
{
    using namespace std;
    cout << "Enter your weight: ";
    float weight;
    cin >> weight;
    string gain = "A 10% increase raises ";
    string wt = boost::lexical_cast<string> (weight);
    gain = gain + wt + " to ";      // string operator()
    weight = 1.1 * weight;
    gain = gain + boost::lexical_cast<string>(weight) + ".";
    cout << gain << endl;
    system("pause");
    return 0;
}
```

我们点击调试的三角符号或者菜单里的run命令，可以看到运行成功了，并且弹出了窗口



总结

我将配置好的项目demo提交到gitee了，感兴趣的可以看一看
项目地址
https://gitee.com/secondtonone1/testvscpp (https://gitee.com/secondtonone1/testvscpp)

浏览(4220) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(564) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足

push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也

降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)